17. Iteração de vetores

Iteração de vetores: percorrendo com for

Guardamos frutas em um vetor e acessamos elementos **um por um**, com índices fixos.

Mas em sistemas reais a lista pode ter 3 ou 3000 itens: ninguém escreve `vetor[0]`, `vetor[1]`, `vetor[2]`... manualmente.

A solução é **iterar**: fazer o `for` caminhar por todos os índices.

O que é iterar

Iterar = passar por cada elemento do vetor, um de cada vez.

O `i` do `for` vira o **índice dinâmico**: em vez de escrever `numeros[0]`, `numeros[1]`..., escrevemos `numeros[i]`, e o valor de `i` muda a cada volta.

```
// índices: 0 1 2 3 4
let numeros = [10, 20, 30, 40, 50];      // length = 5

// índices válidos: de 0 até 4 (length - 1)
for (let i = 0; i < numeros.length; i++) {
    console.log(`numeros[${i}] = ${numeros[i]}`);
}
```

Por que a condição é `i < numeros.length`?

Com 5 elementos, o último índice válido é 4.

Quando `i` chegar a 5, a condição vira false e o laço para: **nunca acessamos um índice inexistente**.

Esse é o padrão universal de iteração.

> **O erro clássico:** usar `i <= numeros.length`. Com 5 itens, isso tentaria acessar `numeros[5]`, que não existe (retorna `undefined`). Sempre use `<` (menor), nunca `<=`.

Ordem crescente: do primeiro ao último

```
div_mensagem.innerHTML = '<b>Ordem crescente:</b><br>';
for (let i = 0; i < numeros.length; i++) {
    div_mensagem.innerHTML += `numeros[${i}] = ${numeros[i]} <br>`;
}
```

`i` percorre 0, 1, 2, 3, 4 → exibe 10, 20, 30, 40, 50.

Ordem decrescente: do último ao primeiro

Para inverter, o `i` começa no **último índice** (`length - 1`) e **diminui até 0**:

```
div_mensagem.innerHTML += '<br><b>Ordem INVERSA:</b><br>';
for (let i = numeros.length - 1; i >= 0; i--) {
    div_mensagem.innerHTML += `numeros[${i}] = ${numeros[i]} <br>`;
}
```

> **Detalhe fino:** a condição é `i >= 0` (com o "="), porque **0 ainda é um índice válido**. Se fosse `i > 0`, o primeiro elemento ficaria de fora!

Resumo visual das duas direções

```
// crescente:  i -> 0, 1, 2, 3, 4  (i++) "do começo ao fim"
// decrescente: i -> 4, 3, 2, 1, 0  (i--) "do fim ao começo"
// i é apenas o contador; quem "pega o valor" é o vetor[i].
```

Aplicação: somar todos os valores

A iteração resolve problemas clássicos. Somar os valores de um vetor é o exemplo mais comum: o segredo é uma **variável acumuladora** (a mesma ideia do `+=` das variáveis) que começa em 0 e vai somando:

```
let soma = 0;                       // acumulador
for (let i = 0; i < numeros.length; i++) {
    soma = soma + numeros[i];       // ou: soma += numeros[i]
}
console.log('Soma:', soma);             // 10+20+30+40+50 = 150
```

- **Carrinho de compras:** somar o preço de cada item → total a pagar;
- **Média da turma:** somar todas as notas e dividir por `notas.length`;
- **Lista invertida:** exibir histórico do mais recente para o mais antigo (ordem decrescente);
- **Gerar relatório:** percorrer os produtos e montar a listagem completa.

> **Resumo: iterar** é percorrer o vetor com o `for`: o `i` vira o índice dinâmico, a condição `i < length` protege o fim do vetor (**nunca** `<=`!). Crescente com `i++`, decrescente com `i--` a partir de `length - 1`. E quando queremos **mostrar só uma parte** dos elementos ("apenas as tarefas pendentes")? O segredo é combinar o `for` (passar por todos) com o `if` (decidir quem entra): o filtro de vetores.